medimg

Compressed Sensing Reconstruction for Accelerated MRI and Sparse-View CT

Technical note

Abd'gafar Tunde Tiamiyu
School of Mathematics, Jilin University
Changchun 130012, Jilin, China
abdgafartunde@yahoo.com

Version 0.2.0, October 2026

DOI: 10.5281/zenodo.21321199

Summary

medimg is an open-source Python package for image reconstruction from undersampled data in two settings: accelerated magnetic resonance imaging (MRI), where part of k-space is sampled, and sparse-view computed tomography (CT), where few projection angles are measured. It provides the forward operators and their exact adjoints, both normalized to unit norm, and three reconstruction methods behind one interface: Tikhonov regularization (conjugate gradients), total variation (Chambolle–Pock), and wavelet compressed sensing (FISTA). This note documents the operators and algorithms as implemented, reports the verification results (dot-product tests, operator norms, orthogonality of the wavelet transform, golden-output tests) and the reconstruction results on test images, gives guidance on choosing the noise model and the regularization parameters, and states the limitations. A step-by-step tutorial is provided separately as examples/00_getting_started.ipynb.

Contents

1 Introduction **2**

2 Forward models **2**
2.1 MRI . . . 2
2.2 CT . . . 3

3 Reconstruction methods **3**
3.1 Tikhonov regularization . . . 3
3.2 Total variation . . . 3
3.3 Wavelet compressed sensing . . . 4
3.4 Image quality metrics . . . 4

4 Verification **5**

5 Results **5**
5.1 The examples . . . 5
5.2 Further studies . . . 5
5.3 Run times . . . 6

6 Choosing the parameters **6**

7 Software reference **7**

8 Limitations **7**

1 Introduction

Both MRI and CT acquire linear measurements $y = Ax + \eta$ of an image x. Acquiring fewer measurements shortens MRI scans and reduces CT dose, but makes A non-invertible, and the direct reconstruction (the zero-filled inverse Fourier transform, or filtered back-projection) shows aliasing or streak artifacts. Compressed sensing [2, 5] shows that an image that is sparse in some representation can be recovered from incoherent undersampled measurements by solving a regularized problem that promotes sparsity; Lustig et al. [6] applied this to MRI, and Sidky and Pan [8] to CT with total variation.

`medimg` implements this setting for two-dimensional single-channel images. Its design is deliberately small: every reconstruction function takes the forward operator `A` and its adjoint `At` as plain Python functions, so the same solvers serve MRI, CT, and any operator a user writes. It depends on NumPy, SciPy, Matplotlib, scikit-image, and PyWavelets.

Installation. From the repository root, `pip install -e ".[dev]"` installs the package with the test and notebook dependencies; `uv sync -extra dev` reproduces the exact environment recorded in `uv.lock`. The tutorial `examples/00_getting_started.ipynb` covers both modalities step by step and is executed by continuous integration on every change.

2 Forward models

Images are real arrays $x \in \mathbb{R}^{n_r \times n_c}$ with the Euclidean inner product. Both operators are linear, have an exact adjoint, and are normalized so that $\|A\| = 1$; regularization parameters are therefore on comparable scales for the two modalities.

2.1 MRI

Operator. Let $\mathcal{F}$ be the unitary two-dimensional discrete Fourier transform with the zero frequency at the centre of the array (`fftshift(fft2(ifftshift(x), norm="ortho"))`), and $M \in \{0,1\}^{n_r \times n_c}$ a sampling mask. Then

$$A_{\mathrm{MRI}}\, x = M \odot \mathcal{F}x, \qquad A^*_{\mathrm{MRI}}\, y = \mathrm{Re}\big(\mathcal{F}^{-1}(M \odot y)\big), \tag{1}$$

where $\odot$ is the entrywise product and k-space carries the real inner product $\mathrm{Re}\langle \cdot, \cdot \rangle$ (`mri_forward`, `mri_adjoint`). Because $\mathcal{F}$ is unitary and M a projection, $\|A_{\mathrm{MRI}}\| = 1$, and A^*_{MRI} applied to the data is the zero-filled reconstruction.

Sampling mask. `generate_cartesian_mask(shape, sampling_rate, calib_lines, rng)` fully samples the `calib_lines` central columns of k-space, which hold most of the energy, and then samples individual locations outside that band, without replacement, with a probability proportional to $(1 - d^2)^2$, where d is the distance of the column from the centre column relative to half the width. The total number of samples is `sampling_rate` times the number of pixels, so the acceleration factor is $1/$`sampling_rate`. Because the random part selects points rather than whole lines, the mask corresponds to a 3-D Cartesian acquisition with two phase-encoding directions; single-slice 2-D MRI acquires whole lines (see Section 8).

Noise. `mri_forward(x, mask, noise_std, rng)` adds complex Gaussian noise whose real and imaginary parts are independent $\mathcal{N}(0, \sigma^2)$, at the sampled locations only: $y = M \odot (\mathcal{F}x + \eta)$. Unsampled locations stay exactly zero. Because $\mathcal{F}$ is unitary, σ is on the scale of the image intensities. To prescribe the data signal-to-noise ratio instead, take

$$\sigma = \frac{\mathrm{rms}\big((\mathcal{F}x)|_M\big)}{\sqrt{2}\, 10^{\mathrm{SNR}/20}}, \tag{2}$$

as the MRI example and the tutorial do.

2.2 CT

Operator. For parallel-beam CT the forward operator is the Radon transform $\mathcal{R}$, sampled at angles $\theta_1, \ldots, \theta_{N_\theta}$ in $[0, \pi)$ and at n unit-width detector bins for an $n \times n$ image. `medimg` discretizes it as a sparse matrix with a pixel-driven scheme: each pixel centre inside the inscribed circle is projected onto the detector, $s = x\cos\theta + y\sin\theta + \lfloor n/2 \rfloor$, and its value is split between the two nearest bins with linear-interpolation weights. The geometry matches `skimage.transform.radon(image, circle=True)`: on the Shepp–Logan phantom the two sinograms differ by 1.3% (relative), owing to the different interpolation. The back-projection is the matrix transpose, so the pair is exactly adjoint. With `normalize=True` (the default) both `ct_forward` and `ct_adjoint` divide by the largest singular value of the matrix, computed once with `scipy.sparse.linalg.svds`, so that $\|A_{\mathrm{CT}}\| = 1$; with `normalize=False` the forward operator returns line integrals in pixel units, as filtered back-projection (FBP) expects. Matrices are cached for the eight most recent combinations of image size and angles.

Version 0.2.0 replaced the earlier operator pair, which applied `skimage.transform.radon` and `iradon` with a fixed normalization: `iradon` scales the back-projection by $\pi/(2N_\theta)$, and the two functions are not discrete adjoints of each other, so the dot-product ratio was 17.6, 40.8, and 86.6 for 30, 60, and 180 angles. Likewise, the earlier MRI operator used the unnormalized FFT with the normalized inverse, which made its adjoint wrong by a factor of $n_r n_c$.

3 Reconstruction methods

All three functions take `A`, `At`, the data `y`, a regularization parameter, and the image `shape`. Where a step size is needed, the Lipschitz constant $L = \|A\|^2$ is estimated by 30 power iterations of A^*A from a random start (`rng` sets the seed) unless `L` is given.

3.1 Tikhonov regularization

`tikhonov_cg(A, At, y, lambd, shape, x0, max_iter, tol)` solves

$$\min_x \ \tfrac{1}{2}\|Ax - y\|^2 + \tfrac{\lambda}{2}\|x\|^2 \tag{3}$$

by conjugate gradients on the normal equations $(A^*A + \lambda I)x = A^*y$, stopping after `max_iter` iterations or when the residual has fallen below `tol` times its value at the previous iteration. For MRI it barely improves on the zero-filled image: it cannot recover frequencies that were not measured, apart from those fixed by the conjugate symmetry of a real image.

3.2 Total variation

`tv_chambolle_pock(A, At, y, alpha, shape, L, max_iter, theta, rng)` solves

$$\min_x \ \tfrac{1}{2}\|Ax - y\|^2 + \alpha\, \|\nabla x\|_{2,1}, \qquad \|\nabla x\|_{2,1} = \sum_{i,j} \sqrt{(\nabla_1 x)_{ij}^2 + (\nabla_2 x)_{ij}^2}, \tag{4}$$

where ∇ takes forward differences with periodic boundary conditions, by the primal-dual method of Chambolle and Pock [3] applied to $K = [A; \nabla]$ (Algorithm 1). Since $\|\nabla\|^2 \le 8$, the steps $\tau = \sigma = 0.99/\sqrt{L+8}$ satisfy $\tau\sigma\|K\|^2 < 1$. The dual variable for the data term has the data's type (complex for MRI, real for CT). The method runs exactly `max_iter` iterations; there is no stopping test.

Algorithm 1 Chambolle–Pock for (4) (`tv_chambolle_pock`)

Require: A, A^*, y, $\alpha > 0$, $L \ge \|A\|^2$, $\theta \in [0,1]$

$\tau = \sigma = 0.99/\sqrt{L+8}$; $x^0 = \bar{x}^0 = 0$; $p^0 = 0$; $q^0 = 0$

for $k = 0, 1, \ldots,$ `max_iter` $- 1$ **do**

  $p^{k+1} = \big(p^k + \sigma(A\bar{x}^k - y)\big)/(1+\sigma)$ ▷ prox of the conjugate of $\frac{1}{2}\|\cdot - y\|^2$

  $q^{k+1} = \Pi_\alpha\big(q^k + \sigma\nabla\bar{x}^k\big)$ ▷ pointwise projection onto $\{|q_{ij}| \le \alpha\}$

  $x^{k+1} = x^k - \tau\big(A^*p^{k+1} - \operatorname{div} q^{k+1}\big)$

  $\bar{x}^{k+1} = x^{k+1} + \theta(x^{k+1} - x^k)$

end for

return $x^{\texttt{max_iter}}$

3.3 Wavelet compressed sensing

`cs_fista(A, At, y, W, Wt, lambd, shape, L, max_iter, tol, rng)` solves

$$\min_x \ \tfrac{1}{2}\|Ax - y\|^2 + \lambda\|(Wx)_{\text{detail}}\|_1 \tag{5}$$

by FISTA [1] (Algorithm 2), where W is a two-dimensional wavelet transform and only the detail coefficients are penalized. `wavelet_forward` and `wavelet_adjoint` use periodic extension (`mode="periodization"`), so for an orthogonal wavelet (Haar, Daubechies, symlets, coiflets) and image sides divisible by 2^{level} the transform is orthogonal and $W^*\mathcal{S}_{\lambda/L}W$ is the exact proximal map of the penalty, as FISTA requires [4]. FISTA stops when the relative change of the iterate falls below `tol`.

Version 0.2.0 introduced the periodic extension. With PyWavelets' default symmetric extension, the db4 transform of a 128×128 image has 21 904 coefficients and is not orthogonal ($\|Wx\|/\|x\| = 1.097$), so the thresholding step was only an approximation of the proximal map; the Haar transform was unaffected.

Algorithm 2 FISTA for (5) (`cs_fista`)

Require: A, A^*, y, W, W^*, $\lambda > 0$, $L \ge \|A\|^2$

$x^0 = z^0 = 0$, $t_0 = 1$

for $k = 0, 1, \ldots,$ `max_iter` $- 1$ **do**

  $x^{k+1} = W^*\,\mathcal{S}_{\lambda/L}\big(W(z^k - \frac{1}{L}A^*(Az^k - y))\big)$ ▷ soft-threshold detail coefficients

  $t_{k+1} = \frac{1}{2}\big(1 + \sqrt{1 + 4t_k^2}\big)$

  $z^{k+1} = x^{k+1} + \frac{t_k - 1}{t_{k+1}}(x^{k+1} - x^k)$

  if $\|x^{k+1} - x^k\| \le$ `tol` $\|x^{k+1}\|$ **then stop**

  end if

end for

return x^{k+1}

3.4 Image quality metrics

`psnr(x, x_ref)` returns $20\log_{10}(\text{range}) - 10\log_{10}\text{MSE}$, with the range of the reference image (max − min) as the peak value. `ssim(x, x_ref)` implements the structural similarity index [9]

with an 11×11 uniform window and $K_1 = 0.01$, $K_2 = 0.03$; its values can differ slightly from implementations that use a Gaussian window.

4 Verification

- *Adjoints.* For random x and y, the ratio $\langle Ax, y\rangle / \langle x, A^*y\rangle$ equals 1 to within 2×10^{-15} for the MRI operator and for the CT operator with 30, 60, and 180 angles (the tests check 17, 45, and 180 angles).
- *Operator norms.* Power iteration gives $\|A\|^2 = 1$ to six digits for both operators.
- *Noise model.* Unsampled k-space locations are exactly zero, and the empirical standard deviation of each noise component matches `noise_std`.
- *Wavelets.* For Haar, db4, and sym8 the transform preserves the norm and passes the dot-product test to 10^{-12}.
- *Solvers.* Chambolle–Pock on real CT data raises no complex warnings and returns a real image.
- *Golden outputs.* `tests/test_golden.py` reruns the quick configuration of both examples and compares the PSNR and SSIM of every method with stored values (`tests/golden/quick_examples.json`).

The test suite has 60 tests. Continuous integration runs it on Python 3.10 to 3.13, runs both examples in quick mode from outside the repository, and executes the tutorial.

5 Results

5.1 The examples

The MRI example, `examples/01_mri_cs.py`, reconstructs the 256×256 Shepp–Logan phantom from 25% variable-density sampling (4× acceleration) at a data SNR of 30 dB. The CT example, `examples/02_ct_sparse_view.py`, uses 45 parallel-beam views with Gaussian noise of 1% of the sinogram RMS (40 dB). The regularization parameters were chosen by a coarse search for the best PSNR on this phantom, so the results are best cases for it. Table 1 lists the metrics and Figures 1 and 2 show the images.

Table 1: PSNR (dB) and SSIM for the 256×256 examples, with the regularization parameters used.

	baseline	Tikhonov	TV	CS (Haar)
MRI, 4×, 30 dB	21.2 / 0.41	22.1 / 0.33	44.0 / 0.99	34.7 / 0.92
parameter	zero-filled	$\lambda = 10^{-3}$	$\alpha = 3 \times 10^{-3}$	$\lambda = 3 \times 10^{-3}$
CT, 45 views, 1%	22.9 / 0.51	25.3 / 0.67	31.9 / 0.99	27.7 / 0.83
parameter	FBP	$\lambda = 3 \times 10^{-4}$	$\alpha = 10^{-3}$	$\lambda = 3 \times 10^{-4}$

5.2 Further studies

The tutorial adds three studies at 128×128:

- *Parameter choice for MRI.* The discrepancy principle [7], which chooses the TV parameter whose residual $\|Ax_\alpha - y\|$ matches the noise norm $\sqrt{2m}\,\sigma$ (m sampled locations), selects $\alpha = 5.6 \times 10^{-3}$ (36.2 dB), against the best value 3.2×10^{-3} (37.5 dB).
- *Number of CT views.* With α retuned for each case, TV reaches 22.9, 27.3, 30.2, and 30.6 dB with 15, 30, 60, and 120 views; the best α falls from 3×10^{-3} to 10^{-4}.
- *A natural image.* On the `camera` image from scikit-image (MRI, same mask and noise), zero-filling gives 23.2 dB, TV ($\alpha = 3 \times 10^{-3}$) 30.2 dB, and Haar CS ($\lambda = 3 \times 10^{-3}$) 27.1 dB;

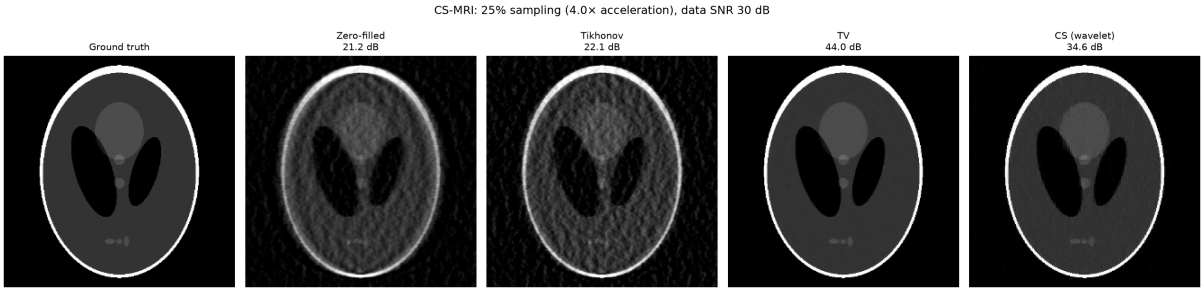


Figure 1: MRI at 4× acceleration and 30 dB SNR.

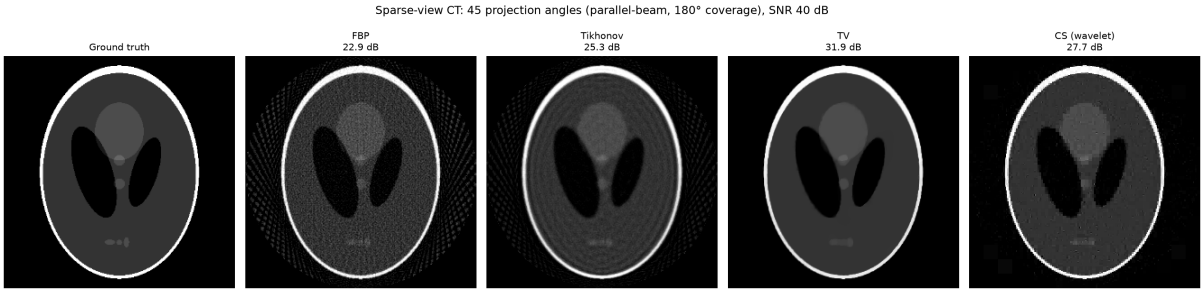


Figure 2: Sparse-view CT from 45 views with 1% noise.

db4 wavelets give about 24 dB. The gain from TV is about 7 dB, against 17 dB on the piecewise constant phantom.

5.3 Run times

On an Apple M-series laptop with Python 3.13, applying A and then A^* to a 256×256 image takes 1.0 ms for MRI and 4.8 ms (45 views) or 20.9 ms (180 views) for CT. Building the CT matrix and its norm takes 0.25 s and 1.0 s for these two cases (4.6 and 18.4 million nonzeros), once per image size and set of angles. The full MRI example runs in about 5 s and the full CT example in about 20 s; the version 0.1 CT example took about eight minutes.

6 Choosing the parameters

Noise. Specify the MRI noise through the data SNR with (2); 30 dB is a moderate level. For CT, the examples add Gaussian noise relative to the sinogram RMS; real CT noise is Poisson and depends on the dose (Section 8).

Regularization parameters. With $\|A\| = 1$ and images in $[0, 1]$, useful values lie roughly between 10^{-4} and 10^{-2} for all three methods. The best value grows with the noise level and falls as the number of measurements grows (Section 5). Without the true image:

- for MRI the model is exact, so the misfit is noise and the discrepancy principle works well when the noise level is known, for example from a noise-only scan;
- otherwise, sweep the parameter on a logarithmic grid and choose where the image stops changing qualitatively; check both PSNR-like and structural measures, which can disagree.

Iterations. Chambolle–Pock has no stopping test: 500 iterations suffice at 128×128 and 1000 to 2000 at 256×256 for the examples. FISTA usually stops earlier on its relative-change test; CG needs at most a few hundred iterations.

Your own operators. Any pair `A`, `At` works. Run the dot-product test first, and either normalize the operator to unit norm or pass `L`; otherwise the solvers estimate $\|A\|^2$ by power iteration and the regularization parameters must be rescaled by it.

7 Software reference

Table 2 lists the public functions.

Conventions. Images are real `float64` arrays. k-space arrays are complex, of the image size, and zero at unsampled locations. Sinograms have shape (N_θ, n). Angles are in radians. CT requires square images, and only the inscribed circle is imaged.

Table 2: Public functions (module prefixes relative to `medimg`).

function	purpose
`forward.generate_cartesian_mask(shape, sampling_rate, calib_lines, rng)`	variable-density sampling mask
`forward.mri_forward(image, mask, noise_std, rng)`, `mri_adjoint(kspace, mask)`	MRI operator pair (1)
`forward.ct_forward(image, angles, normalize)`, `ct_adjoint(sinogram, angles, output_size, normalize)`	CT operator pair
`reconstruct.tikhonov_cg(A, At, y, lambd, shape, x0, max_iter, tol)`	Tikhonov by conjugate gradients
`reconstruct.tv_chambolle_pock(A, At, y, alpha, shape, L, max_iter, theta, rng)`	total variation (4)
`reconstruct.cs_fista(A, At, y, W, Wt, lambd, shape, L, max_iter, tol, rng)`	wavelet compressed sensing (5)
`transforms.wavelet_forward(x, wavelet, level)`, `wavelet_adjoint(coeffs, wavelet)`	orthogonal wavelet transform
`transforms.soft_threshold`, `soft_threshold_coeffs`	soft-thresholding
`phantoms.shepp_logan(shape)`, `circle_phantom(shape, circles, background)`	test images
`metrics.psnr(x, x_ref)`, `ssim(x, x_ref, data_range, K1, K2)`	image quality
`plotting.plot_reconstructions`, `plot_error_map`	plotting

Reproducing the results. The figures in this note are in `figures/` and are regenerated by the two example scripts, which run from any directory and accept `-quick`, `-outdir`, and `-show`. The golden values are regenerated by `python tests/test_golden.py`.

8 Limitations

- *MRI physics.* Single-coil Fourier encoding of a real image: no coil sensitivities (parallel imaging), no image phase (the adjoint returns the real part), and no non-Cartesian trajectories beyond what a custom mask can express.
- *Sampling.* The built-in mask samples individual k-space points outside the calibration band, which suits 3-D acquisitions; for single-slice 2-D MRI, build a mask of whole lines.
- *CT physics.* Two-dimensional parallel-beam geometry on the inscribed circle; no fan or cone beam, no Poisson noise model, no beam hardening.

- *Algorithms.* Chambolle–Pock runs a fixed number of iterations; the TV gradient uses periodic boundaries; the wavelet transform is orthogonal only for orthogonal wavelets and image sides divisible by 2^{level}.
- *Methods.* Classical variational methods only; no learned reconstruction and no GPU support.

References

[1] A. Beck and M. Teboulle. A fast iterative shrinkage-thresholding algorithm for linear inverse problems. *SIAM Journal on Imaging Sciences*, 2(1):183–202, 2009.

[2] E.J. Candès, J. Romberg, and T. Tao. Robust uncertainty principles: exact signal reconstruction from highly incomplete frequency information. *IEEE Transactions on Information Theory*, 52(2):489–509, 2006.

[3] A. Chambolle and T. Pock. A first-order primal-dual algorithm for convex problems with applications to imaging. *Journal of Mathematical Imaging and Vision*, 40(1):120–145, 2011.

[4] I. Daubechies, M. Defrise, and C. De Mol. An iterative thresholding algorithm for linear inverse problems with a sparsity constraint. *Communications on Pure and Applied Mathematics*, 57(11):1413–1457, 2004.

[5] D.L. Donoho. Compressed sensing. *IEEE Transactions on Information Theory*, 52(4):1289–1306, 2006.

[6] M. Lustig, D. Donoho, and J.M. Pauly. Sparse MRI: the application of compressed sensing for rapid MR imaging. *Magnetic Resonance in Medicine*, 58(6):1182–1195, 2007.

[7] V.A. Morozov. On the solution of functional equations by the method of regularization. *Soviet Mathematics Doklady*, 7:414–417, 1966.

[8] E.Y. Sidky and X. Pan. Image reconstruction in circular cone-beam computed tomography by constrained, total-variation minimization. *Physics in Medicine and Biology*, 53(17):4777–4807, 2008.

[9] Z. Wang, A.C. Bovik, H.R. Sheikh, and E.P. Simoncelli. Image quality assessment: from error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4):600–612, 2004.